

Project: Reward Design, Prioritized Sweeping and Generalization Concepts

Project Overview

In this project, you will investigate how changing reward in Reinforcement Learning (RL) can alter the agent's behavior. One of the main aspects you will investigate will be to empirically compare different rewards as well as design rewards yourself. The base algorithm you are expected to implement is the SARSA algorithm we learned in class. You will add a component of prioritized sweeping and will be required to think of ways to make the algorithm "generalize" – more on that later. In the experiment section you will perform sweeps and ablations, comparing the algorithm with and without prioritized sweeping and with and without the generalization mechanism.

Reminder:

Before you dive into the project, it is recommended you refresh your memory on how Gymnasium environments work in general, found in the tutorial in the moodle. You can find basic code.

Library use:

You may use the following libraries in Python:

1. NumPy, Pytorch, SciPy.
2. Gymnasium.
3. Seaborn, matplotlib, pandas, tqdm.
4. Heapq, Queue, PriorityQueue.
5. Python's standard library in general, is OK to use but you should use numpy's random and not the library random in this project.

No other libraries may be used without permission and justification. If you wish to use other libraries you must justify their use. Additional Library use will likely be approved if I think they do not make aspects of the project trivial.

AI use:

For this project, you may use AIs such as ChatGPT and Claude to ask it questions about how the required environment works, RL topics and understanding in general, as well as questions about your code; but if your goal is to become genuinely strong in RL, I highly recommend implementing the high-level logic yourself and using AI mainly for clarification, debugging help only occasionally, and conceptual questions. Experiencing and solving the bugs that arise in smaller RL projects is valuable preparation for larger RL projects. You should add a disclaimer in your report explaining which AI you used, how it was used and whether it generated any part of your code or report and which parts.

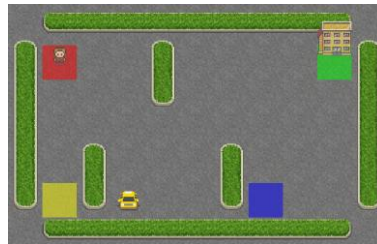
Project Scope:

Below, I will explain the **minimum** requirements for your project. Doing them well can earn you a 100% of the **grade** indeed. However, additional effort can benefit your grade. Therefore, you are encouraged to add as many investigative directions as possible. One useful insight is better than two additional implementations with no insight. I provide some suggestions for additional effort you can put in, labeled "Additional Suggestions". You are permitted and encouraged to think of your own suggestions.

Experimentation Storytelling:

You are required to ground your experimentation in good storytelling. That is, while you are working on your project you will make several decisions, e.g. changing specific parameters which you use in your experiments, or arrive at a particular reward design based on previous observations. This is one of the most important parts of your grading, and I take it quite seriously. You should be able to convince me (in the report as well as the presentation) that you have put significant effort into why your parameter selections have worked as well as how you experienced it throughout experimentation. You should be paying attention to what you are observing so your experiments are grounded in rigor. More information about the report below.

Environment:



For this project, you will use the Taxi environment. This environment already provides you with a base reward. You can read about the environment [here](#). You should set `is_rainy = False`. The environment reward $R_{base}(s, a, s')$ is provided in the documentation. You may use only the default map for training, as Taxi doesn't natively support changing it. You should also use Gymnasium's wrappers if needed to track custom metrics.¹

Additional Suggestions: You can try using multiple gymnasium environments and discuss your algorithm's rigor in the experiment section (below).

Algorithm:

We will use the SARSA algorithm, with an extension from Dyna-Q: Prioritized Sweeping.

As a quick reminder, SARSA uses the following update rule:

$$Q(s_t, a_t) = Q(s_t, a_t) + \alpha \delta_t,$$

where $\delta_t = r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)$.

Dyna-Q stores a table in memory for each observed transition. In class we learned the Prioritized Sweeping Algorithm to use the Q-Learning update, in this project we will instead use a SARSA-style update, requiring your agent to additionally compute the next action while it is planning from its internal model; action selection should be done with ϵ –greedy selection strategy. Be sure not to use the simple Dyna-Q, instead use Prioritized Sweeping. Let the number of planning steps be denoted with n .

¹ Note: You may try setting `is_rainy = True` with varying probabilities as an ablation. Setting `rainy_probability = 1.0`, gets back to deterministic. Doing so, will require you to change how you save transitions in your agent's internal saved environment model during the planning phase (since each state-action pair can see multiple new states). For the update rule in the planning model need to use Expected-SARSA with the kernel K incorporated instead of the Q-Learning update, or you can sample from within your internal model $s', r \sim M(s, a)$.

Algorithm Generalization:

Each SARSA / Dyna-Q update - changes the agent's knowledge for one (s, a) pair. Can we do better? Maybe. The core idea will be that once the agent learns an update to a pair, it should update several other pairs, "seeping" knowledge into similar $(\hat{s}, a)$ pairs.

Fix an observed (s_t, a_t) pair, we will use the following update rule for all $\hat{s} \neq s_t$:

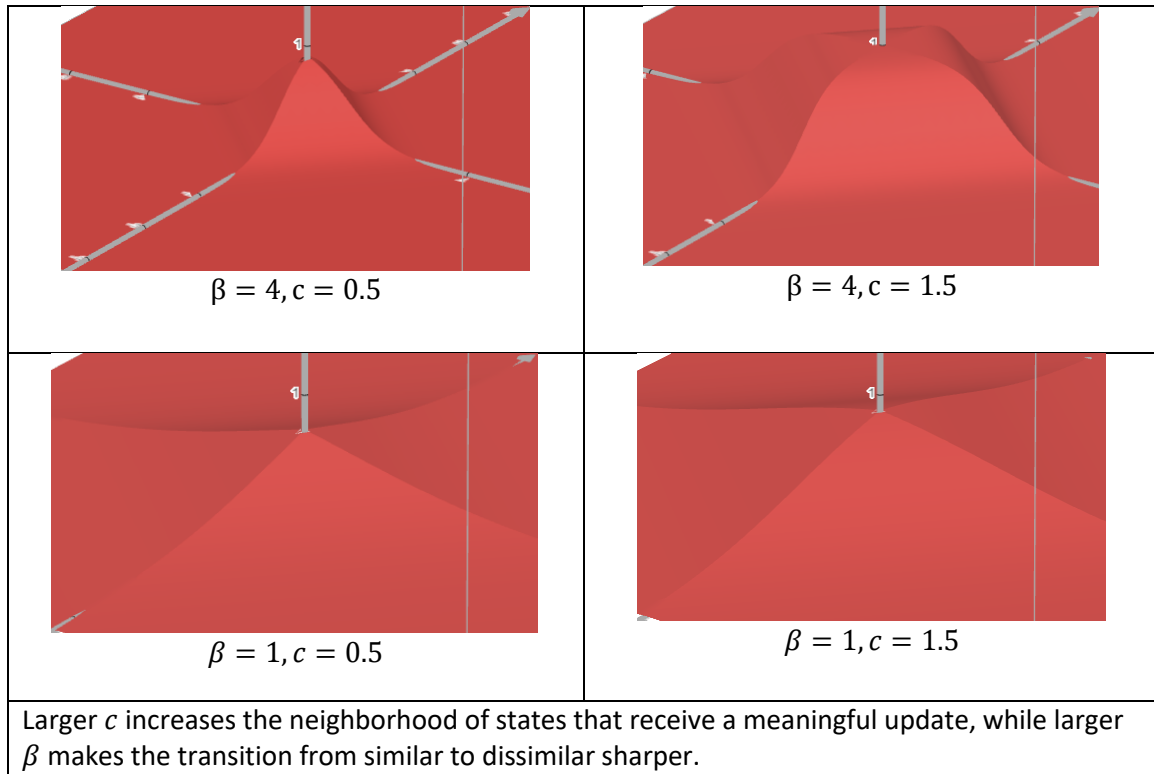
$$Q(\hat{s}, a_t) = Q(\hat{s}, a_t) + \alpha K(s_t, \hat{s}) \hat{\delta}_t,$$

where $K(s, \hat{s})$ measures the **similarity** between the two states². You can try different K functions as long as they are meaningful instead of the suggested function described next.

The suggested function is:

$$K(s, \hat{s}) = K(s, \hat{s}, c, \beta) = \frac{1}{1 + \exp(\beta(d(s, \hat{s}) - c))},$$

where $d(s, \hat{s})$ is the Manhattan distance between the states on the grid, c is the "neighborhood" of very similar states and β is how "sharply" the drop in similarity happens³:



Tip: For the kernel computation, you can either use `numpy.exp(...)`, `scipy.special.expit(...)`, or `torch.sigmoid(...)`.

Tip: Is the planning taking too long? In that case, consider changing it to only compute the Kernel for the 10 or 20 closest states.

² Notice that $\hat{\delta}_t$ is also using the correct trajectory $(\hat{s}, a_t, \hat{r}, \hat{s}', \hat{a}')$. Also please ensure that any Kernel you select is symmetric ($K(s, \hat{s}) = K(\hat{s}, s)$).

³ Note that, c, β are hyper-parameters.

Important: two Taxi states should only be considered similar if they share the same passenger status/location and destination. Otherwise, nearby taxi positions may correspond to completely different subtasks.

Additional Suggestion: (a) Make K decay over time (say a state s has been visited $N(s)$ times) then as the agent visits that state more often ($N(s)$ increases) then its effect over other states should decay. (b) Just as we implemented that states' update "seeps" into other states, we can implement the opposite: only the current (s, a) pair is updated but it "absorbs" the values of nearby states.

Tip: You should implement just one Agent class that accepts arguments specifying whether it uses prioritized sweeping and whether it uses the kernel K , rather than implementing multiple separate classes. Doing so will make implementation quicker, and you will be less prone to bugs in your system.

What is reward design?

Reward design is the process of completely changing the base reward provided by the environment. That is, given a base environment reward $R_{base}(s, a, s')$, we completely ignore $R_{base}(s, a, s')$, and replace it with $R(s, a, s')$ (which is the new reward). In this project, the new reward should not be defined as the base reward plus an extra shaping term. That approach is typically called reward shaping. Here, you are asked to define a complete replacement for the reward function.

What is the point of designing reward?

Reward is an important aspect of the environment's dynamics $\rho(s', r|s, a)$. By changing the base reward we substantially change the MDP in which the agent operates, completely changing the behavior of nearly any useful RL agent and the outcome of learning. Additionally, different reward functions may lead to different performances and thus, designing reward is an important aspect in real applications in RL.

Reward Designs:

You are expected to implement at least four rewards:

1. Base Environment Reward: This is just the reward given by Taxi.
2. Sparse Reward: Give +1 for successful dropoff, +0 everywhere else. Or, +0.5 for successful dropoff and +0.5 for successful pickup.
3. Reward 3: This is a design reward of your choice.
4. Reward 4: This is a design reward of your choice.

In Rewards 3 and 4, you should be creative. In many applications, the reward is the most important aspect, and it tries to combine several "logical" or "reasonable" terms. If you choose a reward that is not easily interpretable but appears to work, you must still make a serious attempt to explain why that happens.

Additional Suggestion: Design more than two rewards. Or, you can add reward shaping terms as an additional thing to compare to.

Experimentation Part 1/2, Reward Comparison:

You are to evaluate the agent's performance across each algorithm setting. Fix $\gamma \in (0,1)$ to some value across all algorithms – this value should not change when comparing performance as it fundamentally changes the underlying MDP, changing it is comparable to adding noise to our

conclusions. Each algorithm has several hyper-parameters (e.g. $\epsilon, \beta, \dots$), you should optimize the algorithm to find out which hyper-parameter set performs best on your reward design. Once you decide on the set, you will compare the four reward designs based on several metrics. You are not required to implement all of the metrics, at least two must be implemented however it is required that you draw conclusions and explain your findings, so your arguments must be convincing based on the evidence you find.

Example metrics include:

1. Cumulative reward through episodes.⁴
2. Success rate over episodes: percentage of episodes ending with a successful dropoff.
3. Average episode length over episodes.
4. Number of illegal pickup/dropoff actions.
5. Percentage of unique states visited each episode. (Each episode the unique states visited is restarted).
6. The mean of the absolute TD error throughout episodes, at each episode n report:
$$\frac{1}{n} \sum_{i=1}^n |\delta_i|.$$

Additional Suggestion: As usual, you can add more metrics. The key here is that metrics should add something **meaningful** to the story.

Experimentation Part 2/2, Parameter Sweeps / Component Ablation:

In this section you will be asked to **ablate different components** or **sweep through parameters** in your algorithm, you are required to sweep at least two parameters and compare ablations across two component setups. You are also required to select metrics to compare the performance between these sweeps/ablations. The metric selection for comparing and evaluating must be grounded in reason, choosing cumulative reward is one metric but certainly not the only one and you should include more.

Select at least two parameters to **sweep** through:

1. $\alpha \in (0,1)$ – the learning rate.
2. $\gamma \in [0,1]$.
3. *rainy_probability* $\in [0,1]$.
4. $\epsilon \in [0,1]$.
5. $\beta \in (0, \infty)$ and/or $c \in (0, \infty)$.
6. $n \in [1, \infty)$, which are the number of planning steps in each iteration.

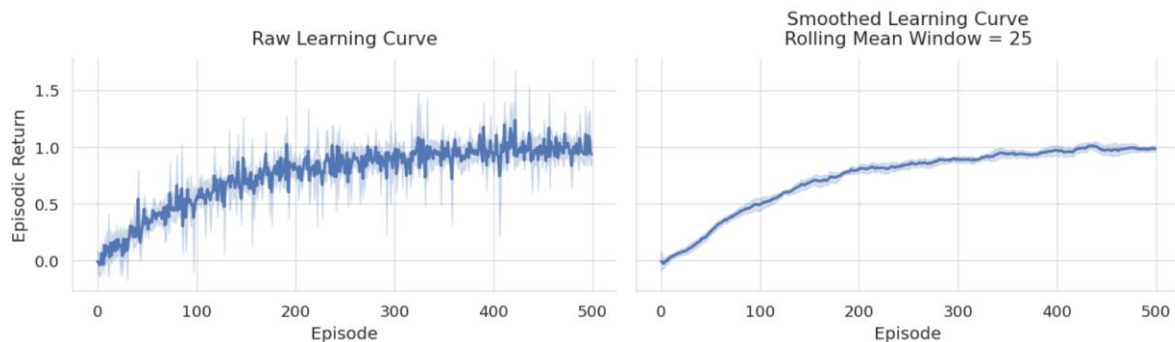
How many values to sweep for each parameter? Depends, make sure to include enough so we can see the trend or explanation you are trying to convey in your story-telling.

The component **ablations** required are to compare the algorithm with/without planning, and with/without the kernel K . There are four possible component setups for that (with/out planning, with/out kernel use).

⁴ It is not always useful to directly compare cumulative reward across different reward designs, since the rewards may have different scales. It is useful however to compare the obtained reward as a performance metric against itself; so questions like the following can be “answered”: is the agent improving as time goes by? Does it get more reward when I use $\epsilon = 0.1$ compared to $\epsilon = 0.01$? etc.

Question: We have several reward designs, which reward designs should see these sweeps / ablations? You should do this for the **base** reward as well as at least one reward which you designed. It should be whichever one (or more) you find interesting. Doing on more rewards may help your grade, but keep the story interesting – that is the important part.

Question: My plots look very noisy, what can I do? You must apply **smoothing** to your noisy plots to make them more readable, but do not add so much that it distorts the conclusion or hides the pattern. In your report, **you must always say when smoothing was applied and how much**. One way to do smoothing is through a “rolling mean”: pandas allows you to do it easily. A small example of smoothing:



Note: **all your experiments** must be done with a seed, so I can re-run your experiments from scratch and obtain the same results. Furthermore, any and all plots you choose to include must have a confidence interval (CI) of 95% displayed across 5 seeds (seaborn makes this easy, if you do more seeds it produces more reliable results).

Additional Suggestion: You can also split the ϵ sweep into ϵ_1, ϵ_2 sweeps where the first one corresponds to action selection during SARSA phase, and the second one corresponds to action selection during the planning phase.

Helpful tips:

These are not required, but only encouraged to help you finish the project faster.

1. While implementing, you may bottleneck over the time it takes for the task to finish. Start small: just one or two episodes. Then, you should use Python’s time library, to find which parts of your code are taking so long so you can optimize it.
2. Consider vectorization early. Numpy supports vectorization, meaning you can compute everything much faster than if you looped over items one by one. Two important places you can do this in this project is at the kernel K , and at the planning phase by computing all planning steps in one go. Here’s a simple example showing the speedup:

```

# A large array of numbers
n = 1_000_000
x = np.arange(n, dtype=float)

# -----
# Without vectorization: Python loop
# -----
start = time.time()

y_loop = []
for value in x:
    y_loop.append(value ** 2 + 3 * value + 1)

y_loop = np.array(y_loop)

end = time.time()
loop_time = end - start

# -----
# With vectorization: NumPy array operation
# -----
start = time.time()

y_vectorized = x ** 2 + 3 * x + 1

end = time.time()
vectorized_time = end - start

# -----
# Compare results and timing
# -----
print("Same result:", np.allclose(y_loop, y_vectorized))
print("Without vectorization:", format_seconds(loop_time))
print("With vectorization: ", format_seconds(vectorized_time))
print("Speedup: ", f"{loop_time / vectorized_time:.2f}x faster")

... Same result: True
Without vectorization: 0.844997644424 seconds
With vectorization: 0.011645793915 seconds
Speedup: 72.56x faster

```

3. You should write tests or just simple print statements for your agent class in different scenarios, since you will be modifying it a lot in the beginning. This way you ensure you're not breaking anything and make development faster.
4. You should use pandas with its ability to write .csv to save the policies you have learned, and you can even use it to save your data for plotting later if the plots don't look exactly as you like. This saves you from re-running the agent learning algorithm just for a plot.
5. Start with the absolute minimum requirements before advancing to more.

Report (At most 20-24 pages, including graphs, text-size=11 or more, text-font=Calibri (Body)):

Prepare a pdf document and in it:

1. AI Use Disclaimer (be explicit on which AI and how you used it).
2. If you did work that is beyond the minimum required, state it clearly right in the beginning, and where I can find it in your report.
3. Your environment / map used for training.
4. Your custom reward designs, and an explanation of how they work, and why one should expect it to work (intuition or formal statements). The kernel you used.
5. A summary over your findings (do this right after showing your custom reward designs). The summary should be insightful into what you found, what worked best and why it worked best grounded in RL explanations. This is just a summary but it is extremely important, it should be convincing. In the next steps, you will expand on the summary and discuss in more detail.
6. Experiment Setup, and algorithm specification ($n = ?$, $\alpha = ?$, ..., etc.).
7. Your best trained policies of each reward design must show an animation over one episode. This must specifically be done using gymnasium's pretty animation display that you see on Taxi's docs.
8. Surprising results you encountered (in terms of agent behavior).

9. Discuss how the Exploration vs. Exploitation problem is encountered across the reward designs.
10. Results section including plots and deep insights, go in depth here.
11. Any pragmatic “tricks” you did to get around any constraints you experienced e.g. time taking too long, memory overloading, etc. You can see the Bandits lecture for tricks to update the average reward while storing only $O(1)$ memory, or use vectorization, etc.
12. Any additional insights you have.

Experiment story-telling: In your report, you must document what you tried and what worked and what didn’t – why you believe the changed caused the observed effect (whether it worked – or didn’t), not everything can be easily explained but even in those cases that statement should be included. Of course it’s difficult to report everything you will encounter, but as a rule of thumb, if it was important to your decision making, it **must** be included. This rule of thumb alone will make your experiment story rich and meaningful.

Further, your report must also include a “Quick Start” section which gives a brief explanation of your project structure, where to find important components, how to navigate them, how to run it, and how to obtain the results you display in your PDF. Very importantly, you must say which libraries and versions you used, or include the correct “pip install” command so your results can be easily checked. They may also be submitted in the form of requirements.txt (if you are familiar). Without the correct library versions, I won’t be able to compile your code. To this end, I highly recommend you use a virtual environment to make things easy for you. See [virtual environment \(venv\) command](#), [W3Schools](#), [Youtube Explanation](#). To keep things as simple as possible use the latest available versions of every library. The “Quick Start” section should be **at most 1 page**.

Your report must also include the project structure – how folders and files are broken up. Please make sure they are logical (e.g. plots can have a folder of their own).

Before you submit your project: try downloading it from scratch on a clean (new) environment, install all the packages you specified in your report with “pip install” (or requirements.txt) and check if you can run it after installing the libraries.⁵

IMPORTANT: Running the entire thing could take a long time, so you should also give me the ability to run with just one seed, rather than the full 5 seeds to reduce wait time by $\times 5$ while checking your submission. It should be an easy command for me to run.

Note: If your report is shorter than 20-24 pages, even if it’s less than 10 pages, it does not mean that that your project is worse than expected, if it is detailed enough to be convincing, that is enough for a full grade. Over-explaining, or repeating statements where it does not add value or information or insight, will not benefit your grade (but it also won’t hurt it).

Presentation:

Your presentation must include:

1. The environment map you used.
2. A brief explanation of SARSA, Dyna-Q and Prioritized Sweeping. The kernel $K(s, s')$ you used and the reasoning behind the kernel you used.
3. The explanation behind your reward designs, grounded in intuition and practical explanation.

⁵ Some students may prefer to do this as a readme.md file and upload the project in the github repo. This is acceptable, but using github is not required.

4. Important: The animations for your policies. It would be interesting to see one policy evolving through training, and the “best” policies for each algorithm running in one slide.
5. Your experiment story-telling – include plots and explain your observations and decision-making. Make sure to explain the plots you are displaying whenever helpful (such as the x and y-axes, the # seeds used in the plots, your hyper-parameter selection). Assume that the students and I will not understand what you are displaying at a moment’s notice when you are explaining a graph.
6. Anything else you feel is worth noting, or found interesting.
7. **Note:** You may be asked to run your code to generate any of the plots you showcased from scratch. Therefore, it should be easy for you to run your code easily and with the right configuration.

Note: Your presentation backs up your report – so your presentation must not contradict your report.

Grading Criteria:

Your project will be evaluated on:

1. Correct implementation of the algorithm.
2. Correct mathematical definition of the reward designs, the Kernel you used.
3. Originality and thoughtfulness of the reward design.
4. Plots: clarity, easy-to-understand x and y axes, whether each plot setting is clear (which parameters were applied, what its evaluating, etc), whether smoothing was applied when it makes the plot look better.
5. Depth of explanation in the report, and how insightful your explanations and experiments are.
6. Your understanding of relevant course materials, including material connected to this project and broader RL concepts discussed in class.
7. Additional work you did, beyond the minimum required.
8. Code quality: the code must be structured well with classes and files that make sense. I need to be able to easily navigate through the code, so variables and classes must be named well and easy to use. As a rule of thumb, I should be able to understand the flow of your code very quickly.
9. Execution quality of everything mentioned in this document.

Good luck! Learning is iterative, and reinforcement learning is especially experimental. Your goal is not only to obtain good results, but to understand how reward design, planning, exploration, and local value propagation affect learning. A major part of this project is to experiment, observe what happens, debug carefully, explain your findings honestly, and enjoy the process.